

Flexible Advancement in Asynchronous BFT Consensus

Shengyun Liu*
Shanghai Jiao Tong University

Wenbo Xu*
Blockchain Platform Div., Ant Group

Chen Shan
Blockchain Platform Div., Ant Group

Xiaofeng Yan
Blockchain Platform Div., Ant Group

Tianjing Xu
Blockchain Platform Div., Ant Group

Bo Wang
Blockchain Platform Div., Ant Group

Lei Fan
Shanghai Jiao Tong University

Fuxi Deng
Blockchain Platform Div., Ant Group

Ying Yan
Blockchain Platform Div., Ant Group

Hui Zhang
Blockchain Platform Div., Ant Group

Abstract

Byzantine fault tolerant (BFT) consensus protocols are becoming an appealing solution to blockchains. As most blockchain systems are deployed on Wide Area Networks (WANs), with each node acting on behalf of its entity, partially synchronous BFT protocols that rely on network synchrony to elect a single leader can be ill-suited. In contrast, asynchronous protocols have no such timing assumptions. Existing asynchronous protocols confront challenges in terms of both flexibility and performance.

Towards enabling adaptability of asynchronous BFT protocols, we propose a new paradigm for bridging *ordering* and *agreement* components. Nodes in the new paradigm can propose and commit blocks in a more flexible manner, in order to suit various workloads in our production environment. To boost performance, we propose SuperMA, an efficient multi-valued agreement protocol. SuperMA in the best case can terminate in three message delays, achieving optimal latency. We further present MyTumbler, a timestamp-based state machine replication protocol. MyTumbler is enhanced by our new paradigm and uses SuperMA as its agreement component. Large-scale experiments on WAN cloud clusters demonstrate the viability of MyTumbler for a wide range of application scenarios.

*Both authors contributed equally.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org. SOSP '23, October 23–26, 2023, Koblenz, Germany

© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0229-7/23/10...\$15.00

<https://doi.org/10.1145/3600006.3613164>

CCS Concepts: • Computer systems organization → Dependable and fault-tolerant systems and networks; • Security and privacy → Distributed systems security.

Keywords: blockchains, Byzantine fault tolerance, asynchronous consensus

ACM Reference Format:

Shengyun Liu, Wenbo Xu, Chen Shan, Xiaofeng Yan, Tianjing Xu, Bo Wang, Lei Fan, Fuxi Deng, Ying Yan, and Hui Zhang. 2023. Flexible Advancement in Asynchronous BFT Consensus. In *ACM SIGOPS 29th Symposium on Operating Systems Principles (SOSP '23)*, October 23–26, 2023, Koblenz, Germany. ACM, New York, NY, USA, 17 pages. <https://doi.org/10.1145/3600006.3613164>

1 Introduction

Byzantine fault tolerant (BFT) consensus protocols are considered a promising solution for both permissioned and permissionless blockchains [19, 39, 42, 47, 79]. A big family of BFT protocols are partially synchronous and leader-based [25, 27, 29, 43, 55, 83], where a single node (i.e., the leader or the primary) is responsible for proposing and sequencing clients' requests.

With rapid growth in the adoption of blockchain systems [3–5, 10], the underlying consensus protocols are facing the following new challenges. First, most blockchain applications are deployed on Wide Area Networks (WANs), where network latency can be high and unpredictable. Second, the number of nodes ranges from dozens to hundreds in permissioned settings [19, 66, 80], while from hundreds to thousands (or even more) in permissionless settings [13, 42]. Third, each node may exclusively serve its customers and may not wish to delegate block generation to other nodes¹. This calls for a highly-efficient, scalable and leaderless protocol.

When confronted with the challenges in blockchain systems, partially synchronous protocols can be ill-suited. First, tuning the maximum network delay Δ in WANs can be a

¹E.g., when the order within a block is critical [84].

tricky problem [7, 38]. A too small Δ may lead to consecutive leader-role changes, while a too large Δ causes a longer delay to recover from failures. Leader-role changes had happened in our cross-border financial applications more often than not, when we used a variant of PBFT [27] as consensus layer. Second, the single leader may become the performance bottleneck [54, 80]. Targeting blockchain deployment, some leader-based protocols rotate the leader-role proactively [25, 83], though they face significant degradation on performance when some nodes are faulty (see our evaluation results in §6.4).

In contrast, *asynchronous* protocols [20, 22, 33, 37, 44, 68] do not rely on timing assumptions and can proceed in a more egalitarian way. Every node acts equally and proposes requests by itself instead of delegating the task to a leader. Each proposal is either *committed* or *aborted*, decided by an agreement protocol that leverages randomization to circumvent the FLP impossibility [40]. Maybe more interestingly, asynchronous protocols also achieve the so-called responsiveness feature [83], meaning that the system always moves forward at the speed of actual network delay. Despite being a more promising direction towards blockchain, existing protocols still suffer from two specific problems:

- every node has to be involved in block proposing in each consensus instance, even if the node at the moment has no request to propose; this happens if the system is under light or unbalanced workloads, which are common in real-world permissioned blockchain applications;
- to circumvent the FLP impossibility, a coin-flipping phase (i.e., randomization) is placed on the critical path of agreement protocols, which may greatly increase time complexity in large-scale deployments [44].

We observe that existing asynchronous protocols use fixed-slot paradigm that pre-assigns a consensus slot to each node. This is not only a waste of resources but also easily introduces competition between proposals: considering f nodes can be faulty, asynchronous protocols allow $n - f$ nodes to quickly push the system forward [41], but this may accidentally abort the proposals of slow nodes. Instead, we target a system that

- can quickly move forward when only one or a few nodes are making proposals at a given moment; and,
- can pace down in order to accommodate more proposals when most nodes are proposing.

Towards this direction, we propose flexible advancement, a new paradigm that allows nodes to adaptively push the system forward, satisfying various workloads in our permissioned blockchain applications. We further present MyTumbler, a timestamp-based state-machine replication (SMR) protocol enhanced by the flexible-advancement paradigm. Timestamp ordering allows each node to position its proposal in a more precise way, rather than compete for the same sequence number. With our new paradigm, MyTumbler

is the first asynchronous BFT SMR protocol that simultaneously achieves the following two features: (1) each node can make its own proposal whenever it has pending requests (in contrast to leader-based protocols where only the leader can propose); and, (2) each node needs to propose only if it has pending requests (in contrast to fixed-slot asynchronous protocols where each node is obliged to propose).

To achieve low latency, we observe that randomization is needed in agreement *only if* correct nodes have divergent opinions on a proposal. We present SuperMA, an efficient multi-valued agreement protocol. SuperMA makes use of a promise mechanism to bypass randomization in common case, and thus can terminate deterministically with only three communication phases. When correct nodes have different views of a given proposal, SuperMA can still go through a coin-flipping phase in order to break ties.

Finally, we implement MyTumbler (using SuperMA) and conduct large-scale experiments on public cloud clusters that span over at most 100 nodes across 4 continents. We also integrate SuperMA into the classical framework of Ben-Or, Kelmer, and Rabin (BKR) [23], which is the core component of HBBFT [68] and other asynchronous protocols [37, 63, 82]. We compare our protocols with three state-of-the-art partially synchronous and asynchronous protocols, i.e., HotStuff [83] under partial synchrony, and BKR and Tusk [33] under asynchrony. Our evaluation results show that when improved by SuperMA, BKR-SuperMA achieved up to an order of magnitude reduction in client-perceived latency. Compared to BKR-SuperMA, MyTumbler further improved peak throughput by more than 2×, achieved even lower latency under unbalanced or light workloads, and accommodated proposals made by far-away node.

2 System Model

We focus on the blockchain or state machine replication (SMR) problem, a solution to which is a fault-tolerant message-passing system that executes requests in a strongly consistent manner. We assume there are a total of n nodes, among which at most $f (< n/3)$ can be *Byzantine faulty*, i.e., exhibit arbitrary behaviors controlled by an adversary. One exception is that the adversary cannot break the cryptographic schemes we use in this paper, including public-key signatures, collision-resistant hash functions and threshold signatures (for coin-flipping phase). We assume each node has a pair of private and public keys, and knows other nodes' public keys in advance. We denote a message m signed by a node i as $\langle m \rangle_{\sigma_i}$. We define the nodes that faithfully follow the protocol and do not crash as *correct* nodes, while the rest are *Byzantine* nodes.

Clients issue requests and send them to nodes. A correct node generates a block proposal by batching requests received. In general, blockchain systems or SMR protocols aim

to commit each proposal by giving it a position, which satisfies linearizability [48] with regard to positions of other committed proposals. Correct nodes then execute requests in committed proposals in a deterministic way. In other words, nodes execute requests sent by clients, in such a way that:

- (Safety) if ES_i and ES_j are the historic execution sequences of two correct nodes i and j at any time, respectively, then either ES_i is a prefix of ES_j or vice versa;
- (Liveness) any request issued by a *correct* client is executed by every *correct* node eventually.

Reliable channel. Each pair of nodes are connected by a reliable channel, namely any message between two correct nodes is eventually delivered. The communication is asynchronous, namely messages between two correct nodes can take arbitrary time to get delivered by the recipient, and can be out of order.

Physical clock. With regard to our protocol (MyTumbler), we assume every node is equipped with a local physical clock. The clocks between correct nodes are loosely synchronized. This can be achieved by external NTP server [1], GPS or built-in clock synchronization protocol [84]. As we will see later, the clock synchronization assumption neither affects the safety nor the liveness of our system, though it can affect the efficiency.

3 Background and Motivation

Characteristics of our permissioned blockchains. We begin by describing several typical scenarios encountered in our production systems. Permissioned blockchain commonly targets deployment in heterogeneous environments, in the sense that the message delays between different pairs of nodes, as well as the workloads (and/or the capacities) of different nodes, are most likely skewed. Each node, on behalf of its entity (e.g., a company or an organization), participates in the maintenance of a distributed ledger. This significantly departs from the settings of traditional SMR protocols, in which each node is presumed to act equally and has equal workload and/or capacity.

First, unbalanced workloads occur in, e.g., our cross-border remittance service. There are e-wallet nodes that process many money transfers from clients, while liquidity provider nodes (usually banks) issue much less requests for making a market. Second, skewed latencies are usually found in geo-distributed deployments, where some nodes are far away from others. For example, our cross-border financial applications contain dozens of nodes across the globe. A super-majority of nodes are located in one continent, while the rest reside in others. Third, researchers often lean towards improvement of peak performance, but maintaining a stable performance (especially client-perceived latency) under a light workload may also be a reasonable goal. For example, in our Business-to-Business e-invoice application, requests are

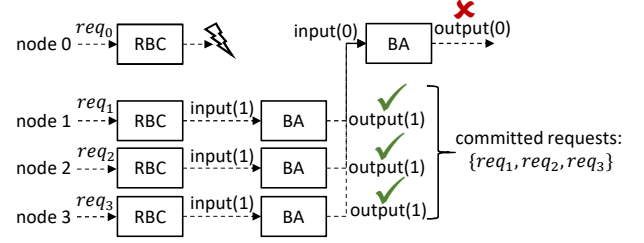


Figure 1. Execution flow of BKR [23, 68] when $n = 4$ ($f = 1$). Request req_0 of node 0 is aborted by other nodes due to the slow network between node 0 and others.

issued infrequently, around several transactions per second. Targeting the scenarios discussed above, neither leader-based nor asynchronous protocols can perfectly fit our needs.

Problems in partially synchronous protocols. Many partially synchronous protocols are leader-based, in which the leader-role is assigned to a fixed node [27, 43, 79] or rotated proactively [25, 83]. At any given point of time, only one node can act as the designated leader. Other (possibly remote) nodes should be aware of the leader location and propagate their cached requests to it. In a nutshell, such protocols have a *dedicated* role in making proposals.

Multi-leader variants [18, 32, 69, 80] assign the leader-role to each node, such that each node can make its own proposals, better utilizing its computing resources and bandwidth. Multi-leader protocols can be considered as running n concurrent single-slot consensus instance in parallel. Even if some nodes have no cached requests at the moment, they have to make an empty proposal in order to push the system quickly forward. In addition, the timing assumptions paid for electing a leader (or a coordinator in multi-leader protocols) can be problematic in the context of geo-replication.

Problems in asynchronous protocols. We first give a brief introduction in the classical framework proposed by Ben-Or, Kelmer, and Rabin (BKR) [23]. BKR is the core component of several asynchronous SMR protocols [37, 63, 68, 82]. The execution flow of BKR is shown in Figure 1. Each BKR instance contains n Reliable Broadcast (RBC) instances and n randomized Binary Agreement (BA) instances, each of which corresponds to a node. RBC allows a proposer to propagate its proposal to others, while BA decides whether the proposal can be committed. RBC ensures *agreement* and *totality*, namely all correct nodes either deliver the same message or all get stuck. RBC does *not* guarantee termination under a faulty proposer, so a BA instance is used to abort the possibly unfinished RBC. Specifically, if a node has delivered a message m in RBC, it inputs 1 to the BA. When $n - f$ BA instances have output 1, the node inputs 0 to the rest BA instances that have no input, in case some f proposers are faulty. Since BA can terminate if every correct node inputs, the output of each BA instance can tell whether a proposal is committed.

Table 1. Comparison of different types of consensus protocols. “Compulsory” means each node is pre-assigned with an agreement slot, which has to be filled with a proposal. “Dedicated” means only one node is responsible for filling slots at a given moment of time.

Compulsory	Dedicated	Typical Protocols
no	yes	PBFT [27], HotStuff [83]
yes	no	BFT-Mencius [69], Mir [80]
yes	no	HBBFT [68], Tusk [33]
no	no	MyTumbler

To summarize, RBC reduces a multi-valued agreement to a binary agreement, while BA is used for termination.

When considering deployment in our production systems, asynchronous protocols like those using BKR are still facing two specific issues: (1) how to better suit all types of workloads; and, (2) how to quickly reach agreement (e.g., finish RBC and BA in BKR).

For the first problem, BKR and other asynchronous protocols require every node to make a proposal, even an empty one when the node has no pending request at hand. This is a waste of resources under light or unbalanced workloads, and $n - f$ fast nodes may suppress proposals of f slow nodes, i.e., making their proposals aborted (e.g., req_0 in Figure 1). This problem is further amplified if we use a fast agreement protocol like SuperMA.

For the second problem, the current implementation for BKR has a big overhead due to the number of communication phases. The RBC of Bracha [24] consists of three communication phases. As for the BA sub-protocol, a good candidate is proposed by Mostéfaoui et al. [71], which also needs three phases of communication in the best case². Moreover, BA of Mostéfaoui et al. puts coin-flipping (i.e., randomization) on the critical path, which makes the best-case latency unlikely to be achieved for the whole BKR procedure [44]. Other optimized protocols [16, 33, 44, 52], though achieving $O(1)$ time complexity in expectation, still put randomization on the execution path, thus hindering fast termination.

Existing protocols pre-allocate a fixed slot for the leader or for each node, and thus have a designated role (dedicated) or require every node (compulsory) to make proposals, as depicted in Table 1. In contrast to the fixed-slot paradigm, we target an approach that can gracefully adapt for different scenarios.

4 MyTumbler

We describe our approach from top to bottom, i.e., in this section we give the flexible-advancement paradigm and MyTumbler, while in §5 we introduce SuperMA.

²The three messages are BVAL, AUX, and an extra CON message to fix a bug [2] related to liveness in HBBFT [68].

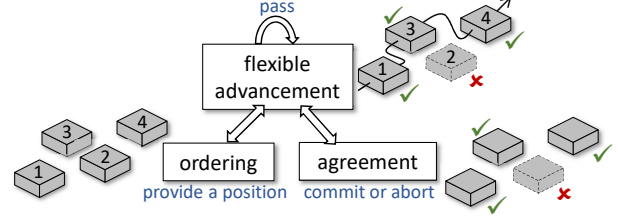


Figure 2. High-level structure of flexible advancement when proposals are ordered by logical sequence numbers.

4.1 The flexible-advancement paradigm

We first investigate how SMR protocols integrate ordering and agreement (i.e., single-slot consensus) components.

SMR protocols proceed by ordering and executing newly-issued proposals. We say a proposal B is *committed* at a position $\phi(B)$, if correct nodes have reached agreement for executing B at $\phi(B)$. Those committed proposals, after agreement, are *totally ordered*. Position $\phi(B)$ can be a sequence number [27, 68], a round number or height in DAG [20, 33], or a timestamp (in MyTumbler). We say the ordering component assigns each proposal to a position.

The agreement component ensures that no two different proposals can be committed at the same position. However, it is not safe to execute a committed proposal immediately. A correct node can safely execute requests in B , *if and only if* B is committed and every committed B' with $\phi(B') < \phi(B)$ has been executed. By continuously executing newly-committed blocks, the system (i.e., the state machine) advances.

One main question for SMR is, how to know all the committed proposals before. In the fixed-slot paradigm, the question is easily answered: every slot must be filled with a proposal. Nevertheless, the drawback is obvious. Even if a node has no requests, it must propose something at a slot.

Actually, *correct nodes only need to know when it is safe to execute a (committed) proposal*, but do not need to fill each slot with a proposal. From the agreement perspective, we observe that a proposal B can be committed at $\phi(B)$, only if a quorum of nodes exclusively *endorse* B . For instance, a proposal in PBFT [27] may be committed, only if $n - f$ nodes have sent PREPARE (or COMMIT) messages for it. In this case, an adversary with f Byzantine nodes cannot find another quorum claiming they have endorsed B' or denying they have endorsed B .

In the new paradigm, we abstract the way correct nodes move forward by requiring each node to explicitly “pass” a position. The idea is inspired by the quorum intersection (for safety) and the fact that any quorum of nodes must be capable of pushing the system forward [41] (for liveness under asynchrony).

The high-level structure of our paradigm is given in Figure 2. To proceed and make newly-committed proposals executable, each node continuously *passes* a new position

$\phi(B)$, meaning that the node *promises* not to endorse any proposal with a position smaller than $\phi(B)$. Once a quorum Q_P of nodes have passed $\phi(B)$, correct nodes know that no newly-issued proposal before $\phi(B)$ can be endorsed by more than $n - |Q_P| + f$ nodes. For the agreement component, we assume a proposal can be committed only if a quorum Q_A of nodes endorse the proposal. If Q_P and Q_A intersect at $f + 1$ nodes, then one correct node is in both groups. Such node should not both have passed $\phi(B)$ and later endorse a proposal before $\phi(B)$. Hence, correct nodes can safely execute all the committed proposals until $\phi(B)$. For simplicity, in this paper we assume symmetric trust³, i.e., $|Q_P| = |Q_A| = n - f = 2f + 1$. We have the following lemma:

Lemma 1. *If $n - f$ nodes claim that they have not endorsed a proposal B , and also promise not to do so in the future, then B cannot be committed.*

With the PASS mechanism, the pace of the whole system is determined by how fast each node passes a new position. Intuitively, if only one node has pending requests (e.g., under unbalanced or light workloads), correct nodes should quickly reach agreement and make the proposal executable, thus achieving low latency. Moreover, such proposal should not be aborted, as in existing asynchronous protocols, the nodes proposing empty proposals may possibly suppress the real workload. On the other hand, when most nodes propose, the system should slow down and accommodate more proposals, thus achieving high throughput.

These two seemingly contradictory modes (fast and slow) can be adaptively and smoothly switched (see §4.2), thanks to the PASS mechanism. One may further refine the restrictions to better fit a particular model, e.g., taking order-fairness into consideration [53, 84]. The designers only need to specify the time for when a position can be passed and the conditions for endorsing a proposal.

The final question is how to determine whether a proposal is committed or aborted, if some correct nodes endorsed it, while others already passed its position. The agreement component is introduced to resolve such ambiguity. In the following, we abstract an agreement component that can be readily used in our paradigm.

Abstraction of ARBC. Multi-valued agreement component can be abstracted as an *Abortable Reliable BroadCast* (ARBC) primitive. Formally, an ARBC instance has a dedicated proposer p . There are three external inputs to the ARBC. First, the proposer inputs and broadcasts a message m . Second, every node may input an *ENDORSE* signal, meaning that the node has received the proposal and agrees to endorse it. Third, every node may input an *ABORT* signal, meaning that the node tries to abort the proposal. As for the output, every node decides a message m or $\perp$, implying that it delivers nothing. The following properties must hold:

- **Agreement:** if two correct nodes decide, they decide the same value (either a message m or a $\perp$).
- **Strong Validity:** if some correct node decides $m \neq \perp$, then m is endorsed by a quorum of $|Q_A| = n - f$ nodes. Moreover, if the proposer is correct and no correct node inputs the *ABORT* signal, then every correct node decides $m \neq \perp$ and m is sent by the proposer.
- **Conditional Termination:** if every correct node inputs the *ABORT* signal, every correct node decides.
- **Totality:** if a correct node decides, eventually every correct node decides.

Upon receiving a proposal B at $\phi(B)$, each node endorses it in ARBC if the node has not yet passed $\phi(B)$. Note that each node may both endorse and (later) try to abort B , in case some correct nodes have already passed $\phi(B)$. The strong validity property is tailored for our paradigm, as no outdated proposal should get committed. As for the conditional termination, when all correct nodes have passed $\phi(B)$, they will be able to terminate all the instances before $\phi(B)$ by inputting the *ABORT* signal. In §5 we give SuperMA, an efficient instantiation of ARBC. In this section, we use the ARBC abstraction to illustrate MyTumbler.

4.2 MyTumbler in a nutshell

Inspired by the success of timestamp-based replication protocols [30, 36, 81, 84], we use physical timestamps as ordering indicators. Intuitively, a block with a smaller timestamp is supposed to be proposed and committed earlier, thus it should not suppress any block proposed at a later time (by trying to abort the corresponding ARBC instance). Sequence number ordering, in contrast, may easily introduce competition for blocks with the same sequence number.

Because nodes in MyTumbler are allowed to propose with non-consecutive timestamps, two questions come up:

- If a node successfully finishes an ARBC instance at timestamp ts , when is it safe to execute its requests? Because of asynchrony, another ARBC instance at $ts' < ts$ may exist, but not yet seen by this node.
- When should a node try to abort an ARBC instance?

The problems are solved with the PASS mechanism. A node can execute the proposal at timestamp ts , only if $n - f$ nodes have passed ts . A node passes ts when it knows $n - f$ ARBC instances (made by distinct proposers) after ts are committed, as it cannot wait forever. Finally, a node inputs an *ABORT* signal for a proposal at timestamp ts if $n - f$ nodes have passed ts but the proposal has not yet been committed.

To enable fast mode, if a node has no request to propose, it simply broadcasts a *SKIP* message, which is considered equally as a committed proposal but does not need to go through the ARBC. A single *SKIP* message cannot guarantee agreement. It only helps correct nodes to pass a timestamp.

When passing a timestamp, each node explicitly broadcasts a PASS message. It has two purposes. First, it serves as

³Flexible advancement may also be applied to asymmetric distributed trust [26, 66], which is out of the scope of this paper.

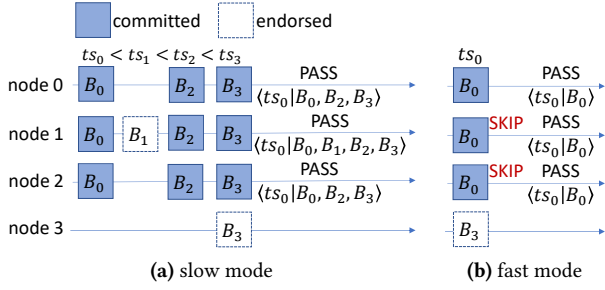


Figure 3. Example for MyTumbler when $n = 4$ and $f = 1$. B_0 , B_1 , B_2 and B_3 are proposed by nodes 0, 1, 2 and 3, respectively. PASS messages also contain the recently endorsed proposals. In Figure 3a, nodes 0, 1 and 2 pass ts_0 upon they commit $n - f$ proposals (including B_0) after ts_0 , making B_0 executable. Timestamp ts_1 is not yet passed by any correct node, and thus B_1 still has a chance to get committed. In Figure 3b, node 0 proposes B_0 at ts_0 , while nodes 1 and 2 skip ts_0 and they all pass ts_0 , making B_0 quickly executable.

a promise that the node will not endorse outdated proposals anymore. Second, it carries the proposals recently endorsed by this node. When a node sees $n - f$ nodes have passed ts , on the one hand, it could learn some proposals that are not known to it. On the other hand, it knows that no other proposal before ts can be committed anymore. As a result, the node just needs to wait until all pending ARBC instances before ts terminate, then all committed proposals before ts can be executed sequentially. An example for MyTumbler is shown in Figure 3.

4.3 MyTumbler in detail

The pseudocode for MyTumbler is given in Algorithm 1. Each ARBC instance is bound to a timestamp ts , a sender ID and a block B , denoted as $MA(ts, sender, B)$. An ARBC instance is *committed* if it decides a non- $\perp$ value, and is *aborted* if it decides $\perp$. The set *endorsed* stores all the instances that the node has recently endorsed, and in *committed* are all the successfully committed instances. Besides, each node keeps track of the latest passed timestamps of all nodes in the array $passd_ts[*]$. Finally, $exec_ts$ is a safety mark. Only proposals that are committed before $exec_ts$ can be safely executed.

When a node i proposes a block B , it broadcasts a signed $\langle \text{PROPOSE}, B, \text{clock}() \rangle_{\sigma_i}$ message using ARBC (Line 2). For the sake of simplicity, each node uses its local system time to make a proposal. Our evaluation results show that this simple strategy can already achieve pretty good performance even with skewed latencies. When node i receives a proposal, it first checks the validity of the timestamp (Line 4). If the timestamp is greater than i 's local time, i will not process the proposal until its local time passes the proposed timestamp. Note that this waiting should not block other operations at i .

Algorithm 1: MyTumbler: Code for Node i .

```

init: endorsed  $\leftarrow \{\}$ , committed  $\leftarrow \{\}$ , passd_ts[*]  $\leftarrow 0$ ,
      exec_ts  $\leftarrow 0$ 
1: upon propose(block  $B$ )
2:   ARBC.broadcast( $\langle \text{PROPOSE}, \text{clock}(), i, B \rangle_{\sigma_i}$ )
3: upon receiving  $\langle \text{PROPOSE}, ts, j, B \rangle_{\sigma_j}$  do
4:   if isValid( $ts$ ) then
5:     initialize an ARBC instance  $MA(ts, j, B)$ 
6:     if passd_ts[ $i$ ]  $< ts$  then
7:       input ENDORSE signal to  $MA$ 
8:       endorsed  $\leftarrow \text{endorsed} \cup \{MA\}$ 
9:   upon an ARBC instance  $MA$  decides  $v$  do
10:    if  $v \neq \perp$  then
11:      committed  $\leftarrow \text{committed} \cup \{MA\}$ 
12:    upon  $\exists ts > \text{passd\_ts}[i]$  s.t.  $n - f$  nodes have their  $MA$ s
      committed and each  $MA.ts \geq ts$  do
13:      ordered-send( $\langle \text{PASS}, i, ts, \text{endorsed} \rangle$ ) to all
14:      passd_ts[ $i$ ]  $\leftarrow ts$ 
15:      endorsed  $\leftarrow \{\}$ 
16:    upon ordered-deliver( $\langle \text{PASS}, j, ts, \mathcal{EB} \rangle$ ) do
17:      for  $\forall MA \in \mathcal{EB}$  do
18:        initialize an ARBC instance
19:        passd_ts[ $j$ ]  $\leftarrow ts$ 
20:        exec_ts  $\leftarrow$  the  $(n - f)^{th}$  highest in passd_ts[*]
21:        for  $\forall MA : MA.ts \leq \text{exec\_ts}$  do
22:          input ABORT signal to  $MA$ 
23:    upon  $\forall MA$  with  $MA.ts \leq \text{exec\_ts}$ :  $MA$  is decided do
24:      execute blocks in committed until exec_ts
25:    upon  $\exists MA$  :
       $n - f$  nodes that passed  $MA.ts$  never endorsed  $MA$  do
26:      let  $MA$  decides  $\perp$ 
27:  function isValid( $ts$ )
28:    if clock()  $< ts$  then
29:      non-blocking wait until clock() =  $ts$ 
30:    return true

```

This mechanism is similar to the idea in Basil [81] or the water mark in PBFT, in case Byzantine nodes purposely propose very large timestamps that exhaust time ranges of correct nodes. The validity check can be extended for protocols that additionally require a validity proof [84], which is out of the scope of this work. Once the timestamp passes the validity check, node i initiates an ARBC instance $MA(ts, j, B)$. Then i compares ts with its own *passd_ts*[i] to determine whether to endorse it (Line 6-8). If the proposal is committed, it is put into *committed* (Line 11).

The PASS mechanism. A node i advances its *passd_ts*[i] to ts as long as $n - f$ distinct nodes have their MA s committed, and each MA has a timestamp after ts . Node i then broadcasts $\langle \text{PASS}, i, ts, \mathcal{EB} \rangle$, where $\mathcal{EB}$ is the set *endorsed*, namely all the recently endorsed MA s since its last PASS message. By doing so, node i promises that it will not endorse any other MA before ts . Then i clears the *endorsed* set (Line 12-15). Upon receiving $\langle \text{PASS}, j, ts, \mathcal{EB} \rangle$ from any node j , the recipient initiates an ARBC instance for each proposal in $\mathcal{EB}$.

and updates $passd_ts[j]$ to ts (Line 16-19). This helps the recipient become aware of proposals it has not seen before.

Although messages can be delivered in arbitrary order, PASS messages must be conveyed through a First-In-First-Out (FIFO) channel between any two correct nodes. This channel can be implemented by assigning sequence numbers to PASS messages plus processing them in order. This ordering restriction is not exclusive to PASS. Other ordering methods such as chain and DAG structures also introduce a similar restriction, albeit in an implicit way.

Advancing $exec_ts$. Each node i advances its safety mark $exec_ts$ to ts , if $n - f$ nodes have advanced their passed timestamps over ts (Line 20). Because at that time, i is informed about all the MAs that can possibly be committed. Node i also tries to abort each instance before $exec_ts$ (Line 22), in case some correct nodes have endorsed the instance, while others already passed it. Node i must wait for the termination of each instance before $exec_ts$. After that, i can execute all committed MA instances until $exec_ts$ (Line 23-24).

For other MAs that appear in none of the $n - f$ PASS messages, they can never be committed because of Lemma 1. Note that an MA can also be aborted if there are $n - f$ nodes claiming that they have passed ts but none of them endorses it (Line 25-26). A malicious node may put an MA only inside the PASS message without proposing it, and send the PASS to a subset of correct nodes. In that case, some correct nodes initiate an ARBC instance for the MA, while others do not know it at all and would not help to terminate it. This special rule guarantees that the MA can still be aborted after all correct nodes have passed $MA.ts$. On the contrary, for any MA that is endorsed by any correct node, eventually every correct node will endorse or try to abort it.

The skipping mechanism. MyTumbler adopts a lightweight skipping mechanism that skips a timestamp if a node i has no pending requests at hand. More specifically, to skip ts , node i broadcasts a $\langle \text{SKIP}, i, ts \rangle_{\sigma_i}$ message. Correct nodes should forward the signed SKIP message to ensure such message is reliably propagated to all. The rule to advance $passd_ts$ (Line 12) is accordingly modified:

- node i advances $passd_ts[i]$ to ts if it sees a group of $n - f$ nodes in which every node j either has an MA committed after ts **or has skipped** $ts' \geq ts$. Note that there must be a committed MA at ts .

A node sends a SKIP when it receives a valid proposal and all of its own requests are already committed. The SKIP need not go through ARBC instance, neither does it need be included in PASS messages.

4.4 Correctness proof

With the agreement and strong validity properties of ARBC, Lemma 1 and ordered delivery of PASS messages, the safety property of MyTumbler can be easily proved, as no correct node will miss any ARBC instance that can be committed.

We sketch the proof of liveness property by the following lemmas.

Lemma 2. *If a correct node i has passed ts , every correct node can eventually pass ts .*

Proof. Node i must have seen $n - f$ committed ARBC instances or SKIP messages from different nodes. Because SKIP is reliably propagated, and ARBC guarantees totality and agreement, other correct nodes can also see those $n - f$ committed ARBC instances or SKIPS, and can pass ts . $\square$

Lemma 3. *If a correct node proposes at ts , every correct node can eventually pass ts .*

Proof. We only need to prove that at least one correct node can pass ts given Lemma 2. Assume by contradiction that no correct node can pass ts , then according to the strong validity of ARBC, the proposal can be committed. For other correct nodes, each of them either proposes an MA with a larger timestamp, or sends a SKIP message. In the former case, the proposal can also be committed because no correct node passes ts . As a result, each of $n - f$ correct nodes either contributes a committed MA or a SKIP. This will enable correct nodes to pass ts , leading to a contradiction. $\square$

Lemma 4. *Every committed MA becomes executable.*

Proof. For any committed MA, either all correct nodes (except the proposer) send SKIP messages for $MA.ts$ and pass $MA.ts$, or at least one correct node proposes with a larger timestamp ts' . In the latter case, every correct node passes ts' (and thus $MA.ts < ts'$) given Lemma 3. Now we need to prove that every pending instance before $MA.ts$ can decide. For any such earlier instance MA' , there are two cases. Case 1: at least one correct node has endorsed MA' . That node will put it into a PASS message and deliver it to everyone. After every correct node passes $MA.ts$, the ARBC conditional termination guarantees that MA' can decide. Case 2: no correct node has endorsed MA' . After correct nodes have all passed $MA.ts$, MA' must decide $\perp$ according to Line 25-26. Thus, every such MA' can decide eventually, making MA executable. $\square$

Lemma 5. *If every correct node proposes a block containing request req , req is committed in some ARBC instance and is executed.*

Proof. It is impossible that all the proposals from $n - f$ correct nodes become aborted. At least $n - 2f$ of them can get committed. Due to Lemma 4, the committed proposal containing req can be executed. $\square$

5 SuperMA: Efficient ARBC Instantiation

A high-level picture of SuperMA is shown in Figure 4. First, we propose a fast negotiation sub-protocol, which on the one hand plays an equal role as a reliable broadcast, i.e., reducing multi-valued agreement to binary agreement. On the other

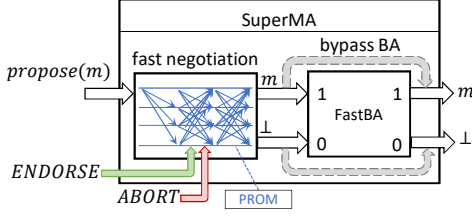


Figure 4. Using fast negotiation to quickly finish the ARBC.

hand, the fast negotiation may totally bypass binary agreement and directly terminate the ARBC. Second, we propose FastBA, a randomized binary agreement (BA) protocol that may also bypass the expensive coin flipping and terminate within two communication phases. FastBA can be simply considered as an extension of the BA protocol of Mostéfaoui et al. [71]. FastBA is used to break ties if correct nodes have divergent opinions about a proposal, i.e., some nodes endorse it, while others try to abort it. FastBA (and other BA protocols) ensure the following properties:

- **BA-Agreement:** if two correct nodes decide, they decide the same binary value $b \in \{0, 1\}$.
- **BA-Validity:** if a correct node decides b , b must be the input of a correct node.
- **BA-Termination:** all correct nodes eventually decide with probability 1.

5.1 The fast negotiation

Intuition. Partially synchronous protocols like PBFT [27] have an efficient message pattern in normal case operations: correct nodes can reach agreement without executing costly view-change operations. Such a feature can be achieved because when the leader (similarly, the proposer in ARBC) is correct and stable, other nodes can receive its proposal and endorse it before timeout occurs (similarly, before correct nodes input the *ABORT* signal). ARBC should also have a fast path that bypasses randomization.

The fast negotiation sub-protocol possesses the very similar message pattern as the normal case operations in PBFT, and has a chance to quickly finish the whole ARBC. More specifically, if a node has obtained a proof that no correct node would input a certain binary value b to the BA, it can immediately decide and output $1 - b$ without waiting for the BA to terminate (due to BA-validity).

The fast negotiation consists of three message phases: *PROPOSE*, *ENDORSE* and (possible) *PROM*.

Propose phase. The dedicated proposer broadcasts m to all (Line 2 in Algorithm 1).

On *ENDORSE*/*ABORT* signal. When receiving the *ENDORSE* or *ABORT* signal (Line 7 or 22 in Algorithm 1), each node i broadcasts a signed $\langle \text{ENDORSE}, h \rangle_{\sigma_i}$ (or $\perp$), where h is the hash digest of m . One exception is if i already entered the BA stage, it does nothing.

Promise phase. Upon receiving $\langle \text{ENDORSE}, h \rangle_{\sigma_s}$ from $n - f$

nodes for the same h , each node i inputs 1 to the BA and propagates the signed *ENDORSE* messages to others. Moreover, if i has *not* sent *ENDORSE* message for $\perp$, it broadcasts a signed $\langle \text{PROM}, h \rangle_{\sigma_i}$, promising it never tried to abort the instance. The case for $\langle \text{ENDORSE}, \perp \rangle_{\sigma_s}$ is similar. Upon receiving $\langle \text{ENDORSE}, \perp \rangle_{\sigma_s}$ from $n - f$ nodes, node i inputs 0 to the BA and propagates the signed *ENDORSE* messages to others. Node i also broadcasts a $\langle \text{PROM}, \perp \rangle_{\sigma_i}$ if i has not endorsed the proposal.

When a node receives $n - f \langle \text{PROM}, h \rangle_{\sigma_s}$, it immediately decides m for the ARBC. This fast path works because in this case no correct node may input 0 to the BA. As an optimization, the node can also propagate the $n - f$ *PROM* messages to help others to quickly decide. Similarly, if a node receives $n - f \langle \text{PROM}, \perp \rangle_{\sigma_s}$ messages, it decides $\perp$.

As the name “fast negotiation” suggests, it is not a full-fledged agreement. Some nodes may input 1 to the BA while others input 0. The BA can solve this divergence.

5.2 The FastBA binary agreement

FastBA is inspired by the BA protocol of Mostéfaoui et al. [71]. In the following we name their algorithm SigFree as it does not use signature (except for random coin).

Both SigFree and FastBA are round-based protocols. In each round, each node votes for one or both binary values. At the end of each round, a common coin, i.e., a random number is generated. The common coin helps nodes converge if their voting values are divergent. SigFree terminates after one or two lucky coin results. Such requirement can be satisfied within $O(k)$ rounds with probability $1 - 2^{-k}$ [68]. Although SigFree has $O(1)$ expected time complexity in communication phases, it always terminates probabilistically. Even if all correct nodes have the same voting values, it still requires one lucky coin, which results in two rounds in expectation. SigFree can be inefficient especially in WANs and large-scale deployment due to the probabilistic termination.

To mitigate this problem, FastBA further bypasses coin flipping when $n - f$ nodes have the same voting value. Specifically, a binary value b can become the final output in SigFree (or FastBA) *only if* b is voted for by $n - f$ nodes. Such strong validity property gives us space for designing a similar promise mechanism. If $n - f$ nodes promise not to vote for $1 - b$ but insist on b , b remains the sole candidate to be decided, even in the presence of f Byzantine nodes. In the best case, FastBA takes only two communication phases to let correct nodes decide. We call this case *fast path*. If correct nodes differ in their voting values, the protocol can still proceed through *normal path*, i.e., random coin. Even in normal path, only one lucky coin is sufficient to bring nodes into the fast path again in the next round. Although digital signature is used, the communication cost is significantly reduced.

The pseudocode for FastBA is given in Algorithm 2. We define a value as *valid* within a round if the value is voted for by $n - f$ nodes. Valid values in round r are stored in $vals_r$.

Algorithm 2: FastBA: Code for Node i .

```

init:  $r \leftarrow 0, vals_* \leftarrow \{\}$ 
1: upon input( $b$ )
2:   broadcast  $\langle BVAL, b, r = 0 \rangle_{\sigma_i}$ 
3: upon receiving  $n - f \langle BVAL, b, r \rangle_{\sigma_s}$  do
4:   if  $b \notin vals_r$  then
5:     insert  $b$  into  $vals_r$ 
6:     re-broadcast these  $n - f \langle BVAL, b, r \rangle_{\sigma_s}$ 
7:   if have not sent  $\langle PROM, *, r \rangle$  or  $\langle AUX, *, r \rangle$  then
8:     if have not sent  $\langle BVAL, 1 - b, r \rangle_{\sigma_i}$  then
9:       broadcast  $\langle PROM, b, r \rangle$ 
10:    else
11:      broadcast  $\langle AUX, b, r \rangle$ 
12: upon receiving  $n - f \langle PROM, b, r \rangle$  do
13:   decide( $b$ )
14: upon receiving  $f + 1 \langle BVAL, b, r \rangle_{\sigma_s}$  do
15:   if have not sent  $\langle PROM, *, r \rangle$  or  $\langle BVAL, b, r \rangle_{\sigma_i}$  then
16:     broadcast  $\langle BVAL, b, r \rangle_{\sigma_i}$ 
17: upon receiving  $n - f \langle PROM, *, r \rangle$  or  $\langle AUX, *, r \rangle$  s.t.
     $vals \subseteq vals_r$  where  $vals$  is the set of values carried by these
     $n - f$  messages do
18:   broadcast  $\langle CON, vals_r, r \rangle$ 
19: upon receiving  $n - f \langle CON, *, r \rangle$  s.t.
     $vals \subseteq vals_r$  where  $vals$  is the set of values carried by these
     $n - f$  messages do
20:    $coin_r \leftarrow \text{getCoin}()$ 
21:   if  $vals_r = \{b\}$  then
22:     if  $coin_r = b$  then
23:       decide( $b$ )
24:     broadcast  $\langle BVAL, b, r + 1 \rangle_{\sigma_i}$ 
25:   else //  $vals_r$  must be  $\{0, 1\}$ 
26:     broadcast  $\langle BVAL, coin_r, r + 1 \rangle_{\sigma_i}$ 
27:    $r \leftarrow r + 1$ 
  run the algorithm until round  $r + 1$  if decide(*) in round  $r$ 

```

Note that only valid values can be finally decided (Line 13, 23), and only valid values can be carried to the next round by correct nodes. This is the key for the promise mechanism in FastBA.

The promise mechanism and fast path. In the first phase, nodes exchange their votes via the BVAL message. Upon receiving $n - f$ votes for the same value b , if a node has not voted for $1 - b$ in this round, it broadcasts a $\langle PROM, b, r \rangle$ message. By doing so, it promises that it will never vote for $1 - b$ in round r . In the second phase, if a node receives $\langle PROM, b, r \rangle$ messages from $n - f$ nodes, it decides b immediately through the fast path (Line 13). The promise mechanism works because in this case $1 - b$ cannot become valid (due to quorum intersection) and b is the only valid option (see Figure 5a for an example).

Normal path. If correct nodes differ in their voting values, FastBA resorts to the normal path that is similar to SigFree (see Figure 5b). In the BVAL phase, each node i also votes for b upon receiving $f + 1 \langle BVAL, b, r \rangle_{\sigma_s}$ from others, so as to ensure at least one value becomes valid (Line 14-16). This is called *amplification*. The only exception is: if node i has promised

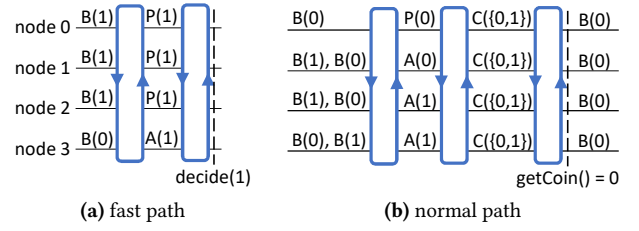


Figure 5. Examples for FastBA. B, P, A and C indicate BVAL, PROM, AUX and CON messages, respectively. In Figure 5a, three nodes promise 1 because they did not send a different BVAL, and each node decides 1 upon receiving $n - f$ PROM(1) messages. In Figure 5b, nodes 1 and 2 first send BVAL(1) and then BVAL(0), while node 3 first sends BVAL(0) and then BVAL(1). In the next round, each node broadcasts the common coin in a BVAL message and goes through the fast path.

the opposite value, it does not amplify. Upon receiving $n - f \langle BVAL, b, r \rangle_{\sigma_s}$, if node i has sent the opposite BVAL for $1 - b$, it broadcasts $\langle AUX, v, r \rangle$ instead of PROM (Line 11). AUX and PROM serve as a notification of the first valid value each node has observed, and can be sent only once per round. After receiving $n - f$ AUX or PROM messages with valid proofs, node i broadcasts a $\langle CON, vals_r, r \rangle$ message reporting its valid value set. Finally, upon receiving $n - f \langle CON, *, r \rangle$ messages, node i can toss a coin and enter a new round (Line 19). There are two cases:

Case 1: if the $n - f$ CON messages contain only one valid value b , i will keep b to the next round. Furthermore, if the coin value happens to be b , i can decide b because it knows that all correct nodes will converge to b in the next round.

Case 2: if the $n - f$ CON messages contain both 0 and 1, i uses the coin value to resolve the ambiguity (Line 26). Because it is a common coin, all such nodes like i will choose the same value to enter the next round.

Complexities. FastBA and SuperMA have $O(\lambda n^3)$ communication complexity and $O(n^2)$ message complexity in expectation, where λ is a security parameter for signatures. Although propagating $O(n)$ signatures introduces higher asymptotic communication complexity, in practice the overhead is relatively small compared to the dissemination of large block data [33].

5.3 Correctness proof

We first prove BA-agreement and BA-termination of FastBA.

Lemma 6. *If nodes i and j report valid-value sets $vals_r$ and $vals'_r$ respectively through CON messages, then $vals_r \subseteq vals'_r$ or $vals'_r \subseteq vals_r$.*

Proof. Proof by contradiction. Assume $vals_r = \{b\}$ on node i and $vals'_r = \{1 - b\}$ on node j . Node i must receive $n - f \langle AUX, b, r \rangle$ (or PROM) messages, say from group $\mathcal{G}$, and node j must receive $n - f \langle AUX, 1 - b, r \rangle$ (or PROM) messages, say

from $\mathcal{G}'$. One correct node is in $\mathcal{G} \cap \mathcal{G}'$, which should not send AUX (or PROM) messages for both b and $1 - b$. $\square$

Lemma 7. *If a node i decides b in round r , then b is the only valid value in subsequent rounds.*

Proof. We discuss two cases. (1) If node i receives $n - f$ $\langle \text{PROM}, b, r \rangle$ and decides b through the fast path, then at least $n - 2f = f + 1$ correct nodes have not sent $\langle \text{BVAL}, 1 - b, r \rangle_{\sigma_s}$. Thus, $1 - b$ can not be a valid value in round r . Only valid values can be voted for by correct nodes in round $r + 1$ (see Line 21-26). (2) If node i decides b through the normal path, according to Lemma 6 and $\text{coin}_r = b$, no correct node will vote for $1 - b$ in round $r + 1$. $1 - b$ cannot be a valid value in round $r + 1$. By induction, $1 - b$ cannot be a valid value in subsequent rounds. $\square$

Theorem 1. (BA-Agreement) *If two correct nodes decide b and b' in round r and r' , respectively, then $b = b'$.*

Proof. When $r = r'$, if b is decided through the fast path, $1 - b$ can not be valid in round r , thus $b = b'$. If both values are decided through the normal path, they must be equal to the same coin value coin_r . When $r < r'$, according to Lemma 7, b is the only valid value in round r' . Thus, $b = b'$. $\square$

Theorem 2. (BA-Termination) *All correct nodes eventually decide with probability 1.*

Proof. We first show that after all correct nodes started a round, they are able to finish that round. Among $n - f$ correct nodes, at least $f + 1$, say group $\mathcal{G}$, must have the same BVAL value b . The remaining correct nodes with value b' form group $\mathcal{G}'$, and they will eventually receive the $\langle \text{BVAL}, b, r \rangle$ messages from $\mathcal{G}$. There are two cases. Case 1: none of the nodes in $\mathcal{G}'$ has promised b' , they will all amplify $\langle \text{BVAL}, b, r \rangle$, making value b valid. Case 2: some node in $\mathcal{G}'$ has promised b' , then it must propagate a quorum certificate of $n - f$ signed $\langle \text{BVAL}, b', r \rangle_{\sigma_s}$ messages, making b' valid. In both cases, every correct node is able to send AUX (or PROM) and further CON messages, and finish this round. By induction, every correct node will keep entering new rounds until termination.

In each round r , the adversary can obtain coin_r only if $f + 1$ correct nodes have sent CON message. According to Lemma 6, each CON message reports either $\{b\}$ or $\{b, 1 - b\}$ for some b . Thus, b will be in vals_r on every correct node when they enter round $r + 1$. With probability $\frac{1}{2}$, $b = \text{coin}_r$ and every correct node only votes for b in round $r + 1$. Through the fast path, they can decide b then. As the round number increases, with probability 1, every correct node decides. $\square$

We then give the proof of the agreement and conditional termination properties of SuperMA, but omit the strong validity and totality proof since it is trivial.

Theorem 3. (ARBC-Agreement) *If two correct nodes decide, they decide the same value (either a message m or $\perp$).*

Proof. There are two cases. (1) At least one node decides x via the fast negotiation. Assume x is a non- $\perp$ message, then at least $n - f$ nodes have promised not to send $\langle \text{ENDORSE}, x' \rangle_{\sigma_s}$ with $x' \neq \text{Hash}(m)$. Thus, none of the correct nodes would send a $\langle \text{PROM}, x' \rangle$, and none of them would fast-decide x' , neither would they input 0 to FastBA. Suppose another node also decides, it either decides m via the fast negotiation, or it obtains output 1 from FastBA. In the latter case, because the decision requires $n - f$ $\langle \text{ENDORSE}, \text{Hash}(m) \rangle$ messages, a different decision m' is impossible. Similarly, if $x = \perp$, then no correct node would send a $\langle \text{PROM}, x' \rangle$ ($x' \neq \perp$), neither would they input 1 to FastBA, making $\perp$ the only possible decision. (2) Both nodes decide via FastBA. The BA-Agreement ensures that they both output 0 or 1. In the former case, they both decide $\perp$. In the latter case, they decide a message m with $n - f$ $\langle \text{ENDORSE}, \text{Hash}(m) \rangle$ messages. $\square$

Theorem 4. (ARBC-Conditional-Termination) *If every correct node inputs the ABORT signal, they can all decide.*

Proof. By the time when every correct node has input the ABORT signal, we also consider two cases. (1) At least one correct node has input some value to FastBA. Because $n - f$ signed ENDORSE messages are also propagated, all the correct nodes are able to input the same value to FastBA, and the BA-Termination ensures that they can all decide. (2) No correct node has entered FastBA. Then they all send a $\langle \text{ENDORSE}, \perp \rangle$, making them able to input 0 to FastBA. $\square$

6 Performance Evaluation

We evaluate the performance of MyTumbler in public cloud clusters and compare to three state-of-the-art protocols, i.e., HotStuff [83] under partial synchrony, and Tusk [9, 33] and BKR [23], the core component of HBBFT [68], under asynchrony. We also integrate SuperMA into BKR (a.k.a., BKR-SuperMA) in order to better demonstrate how SuperMA reduces latency. Tusk [33] is a recently proposed DAG-based asynchronous protocol. In general, Tusk dynamically constructs a DAG in a round-by-round manner. Each round contains blocks made by at least $n - f$ nodes. With the help of common coin, Tusk tries to determine a single block in each odd round as committed. Its ancestors are thus committed recursively.

6.1 Implementation

We have implemented MyTumbler, BKR and BKR-SuperMA in Golang. The hash function we use is SHA256 and the signature algorithm is Ed25519. An open-source implementation of Boneh-Lynn-Shacham (BLS) threshold signature [6] is used to construct a common coin in FastBA. A sequence number is assigned to each PASS message in order to implement the FIFO channel for MyTumbler. Each node processes PASS messages sent by a given peer sequentially based on their sequence numbers. Each node obtains the timestamp from its local physical clock, which by default is synchronized by

the cloud provider. We use the open-source implementations of Tusk [9] and HotStuff [8].

Optimizations. We use batching, the typical technique for boosting throughput. We also use pull-based reliable propagation to improve throughput. Blocks included in PASS messages, i.e., those newly endorsed by a node, only contain the hash of the original requests. Upon a block is committed, if a node has not yet received the original requests, it pulls from others. Other protocols use the similar mechanism except the HotStuff implementation, which propagates requests directly by clients.

6.2 Experimental setup

The platform consists of virtual machines with 8 vCPUs, 32 GiB RAM and 5 Gbps Network burst bandwidth and Ubuntu 20.04.3 LTS. Nodes and clients are deployed over 5 regions across the globe, including Ohio, Singapore, Tokyo, Frankfurt and Canada Central.

For each node, there is a client running in the same region and issuing requests by sending them to nodes in an open-loop manner. In the evaluation of asynchronous protocols (i.e., MyTumbler, BKR, BKR-SuperMA and Tusk), each client sends its requests to the nearby node, which then propagates the requests to others. In HotStuff, every node has all the requests broadcast from each client, thus each block proposed by the leader only contains the hash.

Tusk implementation by default is running in two-process mode. For each node, one process is dedicated to block dissemination, while the other is for all the remaining tasks (i.e., consensus-related messages). Although Tusk and its base structure Narwhal are tailored for scaling out to several machines, we run each node in a single virtual machine for a fair comparison. In MyTumbler, once a SuperMA instance is committed (but may not be executable), the proposer makes a new proposal using the current local time as the timestamp. However, a reply is sent to its client only when the request becomes executable, i.e., we measure the execution latency for all protocols. If a SuperMA instance is aborted, the proposer re-proposes it with a new timestamp.

In our evaluation, we test and stress the protocols with 1000-Byte request, which represents a relatively large transaction in our production system. Also note that 90% of transactions in Bitcoin are below 700 Bytes [11] and the average transaction size in Ethereum is around 560 Bytes [12].

6.3 Fault-free performance

We run the fault-free experiments with different node numbers. We gradually increase the sending speed until it saturates the system. Figure 6 to 8 (FF indicates fault-free) depict the results when $n = 4$, $n = 10$ and $n = 100$ (i.e., $f = 1$, $f = 3$, and $f = 33$), respectively. When $n = 4$, nodes and clients are deployed in Ohio, Singapore, Tokyo and Frankfurt.

As illustrated by these figures, BKR-SuperMA achieves the minimum average latency among all protocols in all configurations. This is because (1) for each of n proposals in BKR-SuperMA, correct nodes can reach agreement through the fast negotiation in three communication phases, i.e., PROPOSE - ENDORSE - PROM messages; and, (2) n SuperMA instances are executed in parallel. These results effectively demonstrate that via SuperMA the latencies of existing asynchronous protocols can be significantly reduced. Although SuperMA introduces $O(\lambda n^3)$ communication complexity, the costs are not significant even in $n = 100$ settings. MyTumbler's latency (MyTumbler-c1 in the figures) is a bit higher compared to BKR-SuperMA because, MyTumbler pays extra costs (i.e., PASS mechanism) for making each proposal executable, while in BKR-SuperMA n proposals can be executed immediately as soon as they all get committed.

Compared to BKR-SuperMA, MyTumbler achieves much higher peak throughput at the cost of moderately higher latencies. This is because (1) timestamp ordering enables MyTumbler to more precisely locate each proposal, and (2) each timestamp is passed only if $n - f$ blocks with higher timestamps are committed, thus making each proposal less likely to be aborted. For example, in Figure 6, when BKR-SuperMA is saturated (point I), 19.7% of its SuperMA instances are aborted. In contrast, when achieving the similar throughput (point II), MyTumbler has no instance aborted. Even when MyTumbler is saturated (point III), only 0.8% instances are aborted.

Tusk's peak throughput is higher than other protocols when n is small ($n = 4$ or $n = 10$). We observe that in WAN settings, more connections between each pair of nodes (e.g., Tusk two-process mode) can effectively increase network performance, as also discussed in [80]. We thus further test the two-connection version of MyTumbler (as denoted by MyTumbler-c2), which achieves the highest peak throughput among all protocols. The latency of Tusk varies from 2.5 to 3 seconds until the system is saturated. Tusk still puts a coin-flipping phase on the execution path, as it commits a DAG round if the next two rounds are finished and a valid block is selected by random coin.

Maybe more interestingly, MyTumbler and BKR-SuperMA achieve lower latencies than HotStuff, even at the beginning of each test. The rotating-leader in HotStuff takes six message phases to commit a block. In contrast, clients in MyTumbler and BKR-SuperMA send their requests to the closest node, which directly makes a proposal and in the best case finish in three communication phases.

There is a significant increase in latency of BKR when running experiments at large-scale (Figure 8). The SigFree algorithm in BKR also puts a coin-flipping phase on the critical path, which grows the expected number of rounds needed for termination. This observation further demonstrates the effectiveness of bypassing randomization.

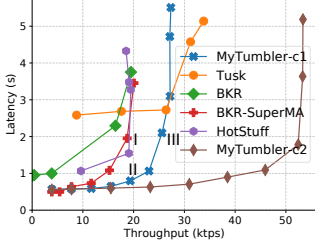


Figure 6. $n = 4$, FF.

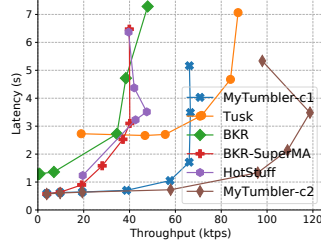


Figure 7. $n = 10$, FF.

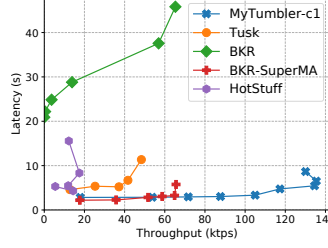


Figure 8. $n = 100$, FF.

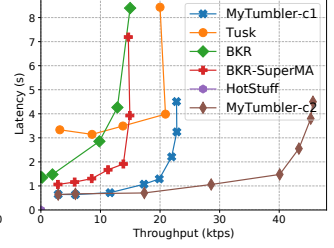


Figure 9. $n = 4$, 1 fault.

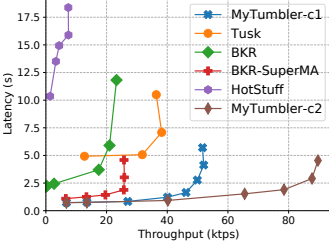


Figure 10. $n = 10$, 3 faults.

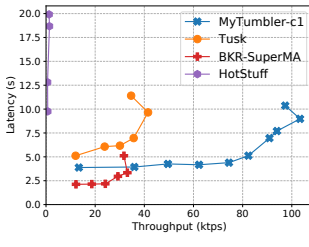


Figure 11. $n = 100$, 33 faults.

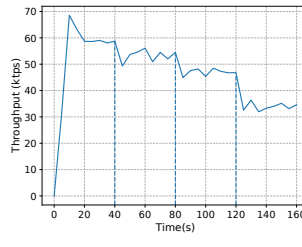


Figure 12. Crash case ($n = 10$).

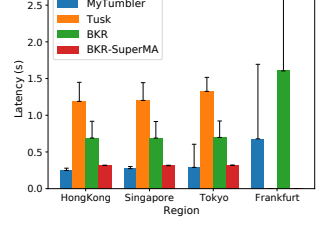


Figure 13. Performance with skewed latencies.

6.4 Performance under faults

Under crash faults. To assess the impact when some nodes are faulty, we first measure the performance when disabling f nodes. In HotStuff, we set the timeout value to 5 seconds, which is the default value in the open-source implementation. When $n = 4$, the node in Ohio is disabled. The results for $n = 4$, $n = 10$ and $n = 100$ are shown in Figure 9-11.

MyTumbler has the minimum average latency when $n = 4$ and $n = 10$. BKR-SuperMA in this case has to deliver $\perp$ for slots assigned to faulty nodes. The slots are processed only after $n - f$ SuperMA instances deliver a non- $\perp$ value. MyTumbler, in contrast, can still smoothly advance passed and executable timestamp as if no node crashed, so long as $n - f$ nodes are correct. The latency of Tusk also increases compared to the fault-free case. In Tusk, the coin-flipping phase randomly selects a block as committed in each odd round. When f nodes crashed, with probability f/n , a null block that should be proposed by a crashed node is selected, which commits no request but delays consensus.

We have also observed that when $n = 4$ and $f = 1$, HotStuff cannot make progress anymore. The problem comes from the pipelining optimization introduced for improving throughput. When $n = 10$, the latency of HotStuff increases dramatically. Since the leader is changed block by block, every correct node has to wait for timeout (5 s in our settings) whenever a crashed node is elected as the new leader. This problem further demonstrates that asynchronous protocols that possess the responsiveness feature are more robust with node failures.

Figure 12 shows the evolution of the throughput of MyTumbler when $n = 10$ and three nodes crash at 40 s (Ohio), 80 s (Tokyo) and 120 s (Canada). The throughput degrades

every time a node crashes. This is because the crashed nodes cannot propose any block and their bandwidth cannot be utilized. There is no fail-over time after each crash, as asynchronous protocols have no timeout for failure recovery.

Under Byzantine faults. Byzantine nodes may badly affect the performance of MyTumbler by trying to disable the fast negotiation. Recall that, the fast negotiation is enabled in SuperMA when most nodes are in consistency about a proposal. Byzantine nodes in this experiment only disseminate their proposals to a subset of f correct nodes and do not participate in message exchange of any proposal. By doing so, problematic proposals are first endorsed by f correct nodes and eventually get aborted, but no one can abort them through the fast negotiation.

We run the experiment with $n = 10$, where two nodes in Frankfurt and one node in Canada Central are Byzantine faulty. Figure 14 shows the results⁴. When the system is not saturated (Figure 14a), the average latency only moderately increases. The peak throughput (Figure 14b) is close to the one with f crashed nodes. Though correct nodes cannot reach agreement through the fast negotiation for Byzantine nodes' proposals, they can still get the output 0 (i.e., abort) in the first round of FastBA (i.e., binary agreement). In particular, the coin-flipping phase is needed in FastBA only if both binary values are voted for by some correct nodes. Such situation may occur frequently if Byzantine nodes can precisely control message scheduling, which we consider can hardly be initiated.

⁴There was a several-month gap between this experiment and others.

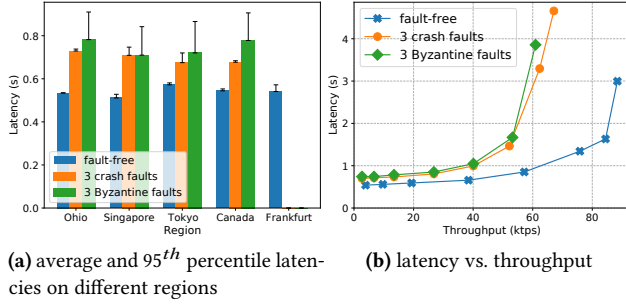


Figure 14. Performance of MyTumbler under Byzantine faults ($n = 10$).

6.5 Latencies in more realistic scenarios

We compare the average latencies under three typical workloads that properly characterize the real-world scenarios.

Far-away node. In this setting, we target a scenario in which a single node is distanced from others, which simulates the settings of our cross-border financial applications. We run the experiment with $n = 4$ nodes and regions: Hongkong, Singapore, Tokyo and Frankfurt. Each client issues requests to its nearby node (5K requests per second). The results are depicted in Figure 13. Since blocks proposed by the node in Frankfurt have to travel over a long distance to get endorsed, the latencies of MyTumbler and BKR in Frankfurt are doubled compared with that of the other regions. Notably, no block proposed by the node in Frankfurt is committed in Tusk and BKR-SuperMA. This is because when the three close nodes see the proposal from Frankfurt, they already entered a new round or decided 0 for the slot assigned to Frankfurt, thus refusing to endorse it. With more skewed distribution, far-away nodes in MyTumbler can further add an adjustment to their timestamps, which we leave for future work.

Light workload. The light workload mimics Business-to-Business e-invoice application scenario. Requests in this case are issued infrequently, around several transactions per second. Each client issues a single request after waiting a uniformly randomized time. The expectation of the interval is set to one second. We run the experiment with $n = 4$ and $n = 10$. The average and 95th percentile latencies are shown in Figure 15. MyTumbler has the lowest latency, mainly because of the skipping mechanism that quickly passes a timestamp and the promise mechanism in SuperMA that bypasses randomization. BKR, BKR-SuperMA and Tusk, even under the light workload, can terminate only if at least $n - f$ blocks are constructed (and committed in BKR or BKR-SuperMA) and/or the coin-flipping phase finishes.

Unbalanced workload. The unbalanced workload characterizes a Cross-Border Remittance Service. We run the experiment in five regions for $n = 10$ nodes, where clients in one or three regions issue requests to its nearby e-wallet server node, at the speed of 5K requests per second. Because

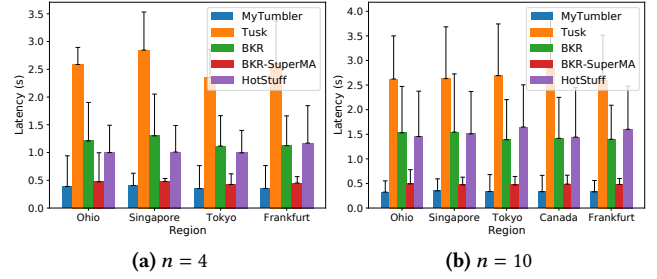


Figure 15. Average and 95th percentile latencies under light workload.

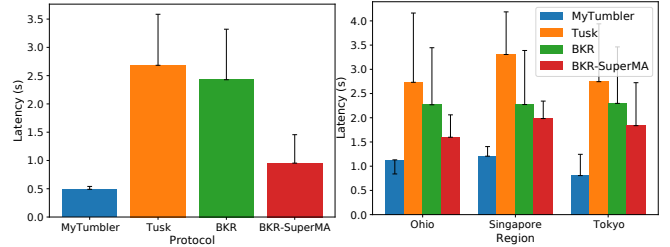


Figure 16. Performance with unbalanced workload ($n = 10$).

there are less than $n - f$ active nodes, the idle nodes have to propose empty blocks and get committed in Tusk, BKR and BKR-SuperMA. The results are depicted in Figure 16. MyTumbler outperforms other asynchronous protocols because of the similar reason: idle nodes quickly skip timestamps for achieving fast passing, while other protocols have to reach agreement for empty blocks.

7 Related Work

7.1 From ordering perspective

Existing ordering methods can be roughly divided into the following four categories.

Sequence number. Since the seminal works of Paxos [56] and PBFT [27], numerous protocols order proposals based on pre-defined sequence numbers [25, 29, 32, 43, 44, 55, 67, 68, 80]. They try to fill out each sequence number, a.k.a., consensus slot, with a batch of requests. Requests are executed sequentially according to their sequence numbers.

Chain. Owing to the prevalence of cryptocurrency such as Bitcoin [72], its chain structure is becoming an appealing solution [15, 42, 47, 83]. Chain-based ordering has no explicit ordering indicator. Instead, nodes propose a block by directly linking it to a previous one. The order of blocks is then derived by the chain structure.

DAG. Instead of linking to a single block, each proposal in DAG protocols contains references to multiple ancestors [20, 33, 52, 60, 61, 78]. As a partial order structure, DAG-based ordering does not have an intrinsic total order, but drives it

by, e.g., dividing blocks into a round-by-round structure and selecting at least one block in some round as committed [20, 33]. Whenever a block is committed, all of its ancestors are committed and totally ordered, breaking ties typically by their hash values.

Timestamp. Timestamp-based protocols order requests according to timestamps attached to each proposal [30, 36, 65, 84]. Pompe [84], a recently proposed work that targets order fairness, leverages timestamps provided by a group of $2f + 1$ nodes to obtain a trusted timestamp. Pompe is designed under partial synchrony and relies on timing assumptions for liveness and fairness properties. Basil [81] is a transactional BFT key-value store that allows each client to drive coordination of transaction processing and choose a timestamp for each transaction it begins.

The flexible-advancement paradigm can be readily integrated into any of those ordering methods except chain-based ordering, since chain structure does not trivially support concurrent proposing by different nodes.

7.2 From agreement perspective

With regard to agreement, i.e., single-slot consensus like SuperMA, we contextualize existing work based on their timing assumptions.

Asynchrony. Protocols [14, 33, 37, 44, 52, 68] under asynchrony do not rely on timing assumptions for safety or liveness. Instead, they resort to randomization to circumvent the FLP impossibility [40]. SuperMA further leverages a promise mechanism to bypass randomization in common case.

Partial synchrony. Protocols in this model rely on synchrony assumption for ensuring liveness, but can guarantee safety despite network condition. Most partially synchronous protocols [25, 27, 31, 43, 55, 83] are leader-based. There are also multi-leader variants, which either pre-partition sequence numbers across nodes [18, 65, 67, 80] or exploit request commutativity [57, 70]. They must rely on timing assumptions to replace problematic nodes. SuperMA does not rely on timing assumptions, thus can terminate even in asynchrony settings.

Synchrony. Protocols that rely on timing assumptions for ensuring safety are collectively named synchronous protocols [15, 35, 45, 58, 64, 75]. SuperMA ensures safety by assuming two-thirds of nodes are honest (i.e., quorum intersection), without additional timing assumptions.

7.3 Other related work and limitations

DispersedLedger [82] decouples a verifiable block dissemination phase from consensus, such that low-bandwidth nodes can still participate in agreement phase even though they have not yet received the blocks. Kauri [73] leverages a tree-based structure to disseminate proposals and aggregate votes to/from large-scale nodes, in order to sustain high throughput for HotStuff. HQ [31] combines a lightweight quorum-based protocol (leaderless) with a BFT SMR protocol, such

that it can perform well both when there is contention and in contention-free case. These approaches are orthogonal to our contributions. Some protocols [17, 21, 28, 50, 59] leverage trusted hardware to prevent equivocation, so as to provide safe and available services with only $n = 2f + 1$ nodes. The flexible-advancement paradigm can also be applied to this model, though in this case $n - f = f + 1$ nodes must be capable of advancing. Eve [51] and Rex [46] exploit multi-core architecture and execute requests concurrently before consensus is achieved. This direction is related to ordering and will be exploited as a future work, since leaderless protocols introduce new challenges for (efficient) deterministic execution. There are also consensus protocols [49, 62, 76] or permissioned blockchain systems [77] that target deployment in datacenter networks. They typically exploit specialized hardware in order to improve performance. Their designs may also be combined with ours, but one must pay careful attention to tolerating Byzantine faults [77] and/or removing the pre-selected leader. Rabia [74] demonstrates that an asynchronous protocol can achieve promising performance even when deployed within a data center.

Limitations. One limitation of MyTumbler and most randomized protocols is the common-coin implementation. We can use threshold signature, but it requires distributed key generation [34] (DKG) if no trusted party is available. MyTumbler also trades off communication complexity ($O(n^3)$ per SuperMA instance) for quick termination. As demonstrated in our experiments and discussed in Tusk [33], the higher complexity may not be significant compared to payload propagation. We can further reduce the communication complexity by using threshold signature and/or gossip dissemination when n is very large.

8 Conclusion

Towards design of flexible and efficient asynchronous BFT protocols, in this paper we introduced MyTumbler and SuperMA. SuperMA is an agreement protocol that reaches consensus on the content of each block proposal, while MyTumbler is a timestamp-based BFT SMR protocol enhanced by the flexible-advancement paradigm, which tells when it is safe to execute a committed block. We only exploited timestamp-based ordering method, though other methods such as sequence-based and DAG-based ordering can be plugged in. As an agreement protocol, SuperMA can be readily used in other asynchronous protocols. One may also apply flexible advancement in CFT model.

Acknowledgments

We are very grateful to Manos Kapritsos (our shepherd) and the anonymous reviewers for their insightful comments. We also thank Rongji Huang and Xiangzhe Wang for their help in developing baseline systems. This work was supported by the Ant Group Research Fund.

References

- [1] 2017. Amazon Time Sync Service. <https://aws.amazon.com/blogs/aws/keeping-time-with-amazon-time-sync-service/>.
- [2] 2018. Bug in ABA protocol's use of Common Coin. <https://github.com/amiller/HoneyBadgerBFT/issues/59>.
- [3] 2019. Walmart turns to blockchain (and Hyperledger) to take on food traceability and safety. <https://www.hyperledger.org/blog/2019/02/21/walmart-turns-to-blockchain-and-hyperledger-to-take-on-food-traceability-and-safety>.
- [4] 2020. XOX partners with AntChain to enhance food traceability and safety for Oversea Enterprise. <https://www.businesstoday.com.my/2020/09/28/xox-partners-with-antchain-to-enhance-food-traceability-and-safety-for-oversea-enterprise/>.
- [5] 2021. Axie Infinity. <https://whitepaper.axieinfinity.com/>.
- [6] 2021. BLS threshold signature. <https://github.com/herumi/bls-eth-go-binary>.
- [7] 2021. Etcd Tuning. <https://etcd.io/docs/v3.4/tuning/>.
- [8] 2021. HotStuff Rust implementation. <https://github.com/asonnino/hotstuff/tree/d771d4868db301bcb5e3deaa915b5017220463f6>.
- [9] 2021. Narwhal and Tusk implementation. <https://github.com/facebookresearch/narwhal>.
- [10] 2021. Uniswap. <https://uniswap.org/>.
- [11] 2022. Bitcoin Transaction Size. <https://bitcoinvisuals.com/chain-tx-size> Accessed: 2022-11-27.
- [12] 2022. Ethereum Statistics. <https://ycharts.com/indicators/sources/etherscan> Accessed: 2022-11-27.
- [13] 2023. Ethereum Mainnet Statistics, total nodes:7702. <https://www.ethernodes.org/> Accessed: 2023-04-17.
- [14] Ittai Abraham, Marcos K. Aguilera, and Dahlia Malkhi. 2010. Fast Asynchronous Consensus with Optimal Resilience. In *Distributed Computing*, Nancy A. Lynch and Alexander A. Shvartsman (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 4–19.
- [15] I. Abraham, D. Malkhi, K. Nayak, L. Ren, and M. Yin. 2020. Sync HotStuff: Simple and Practical Synchronous State Machine Replication. In *2020 IEEE Symposium on Security and Privacy (SP)*. 106–118.
- [16] Ittai Abraham, Dahlia Malkhi, and Alexander Spiegelman. 2019. Asymptotically Optimal Validated Asynchronous Byzantine Agreement. In *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing (Toronto ON, Canada) (PODC '19)*. Association for Computing Machinery, New York, NY, USA, 337–346.
- [17] Marcos K. Aguilera, Naama Ben-David, Rachid Guerraoui, Antoine Murat, Athanasios Xyglis, and Igor Zablotchi. 2023. UBFT: Microsecond-Scale BFT Using Disaggregated Memory. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (Vancouver, BC, Canada) (ASPLOS 2023)*. Association for Computing Machinery, New York, NY, USA, 862–877.
- [18] Salem Alqahtani and Murat Demirbas. 2021. BigBFT: A Multi-leader Byzantine Fault Tolerance Protocol for High Throughput. [arXiv:2109.12664](https://arxiv.org/abs/2109.12664) [cs.DC]
- [19] Elli Androulaki, Artem Barger, Vita Bortnikov, Christian Cachin, Konstantinos Christidis, Angelo De Caro, David Enyeart, Christopher Ferris, Gennady Laventman, Yacov Manevich, Srinivasan Muralidharan, Chet Murthy, Binh Nguyen, Manish Sethi, Gari Singh, Keith Smith, Alessandro Sorniotti, Chrysoula Stathakopoulou, Marko Vukolić, Sharon Weed Cocco, and Jason Yellick. 2018. Hyperledger Fabric: A Distributed Operating System for Permissioned Blockchains. In *Proceedings of the Thirteenth EuroSys Conference (Porto, Portugal) (EuroSys '18)*. Association for Computing Machinery, New York, NY, USA, Article 30, 15 pages.
- [20] Leemon Baird. 2016. The swirlds hashgraph consensus algorithm: Fair, fast, byzantine fault tolerance. *Swirlds, Inc. Technical Report SWIRLDS-TR-2016 1* (2016).
- [21] Johannes Behl, Tobias Distler, and Rüdiger Kapitza. 2017. Hybrids on Steroids: SGX-Based High Performance BFT. In *Proceedings of the Twelfth European Conference on Computer Systems (Belgrade, Serbia) (EuroSys '17)*. Association for Computing Machinery, New York, NY, USA, 222–237.
- [22] Michael Ben-Or and Ran El-Yaniv. 2003. Resilient-optimal interactive consistency in constant time. *Distributed Computing* 16, 4 (2003), 249–262.
- [23] Michael Ben-Or, Boaz Kelmer, and Tal Rabin. 1994. Asynchronous Secure Computations with Optimal Resilience (Extended Abstract). In *Proceedings of the Thirteenth Annual ACM Symposium on Principles of Distributed Computing (Los Angeles, California, USA) (PODC '94)*. Association for Computing Machinery, New York, NY, USA, 183–192.
- [24] Gabriel Bracha. 1987. Asynchronous Byzantine agreement protocols. *Information and Computation* 75, 2 (1987), 130–143.
- [25] Ethan Buchman, Jae Kwon, and Zarko Milosevic. 2018. The latest gossip on BFT consensus. *CoRR* abs/1807.04938 (2018). [arXiv:1807.04938](https://arxiv.org/abs/1807.04938)
- [26] Christian Cachin and Björn Tackmann. 2020. Asymmetric Distributed Trust. In *23rd International Conference on Principles of Distributed Systems (OPODIS 2019) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 153)*. Dagstuhl, Germany, 7:1–7:16.
- [27] Miguel Castro and Barbara Liskov. 1999. Practical Byzantine Fault Tolerance. In *Proceedings of the Third Symposium on Operating Systems Design and Implementation (New Orleans, Louisiana, USA) (OSDI '99)*. USENIX Association, USA, 173–186.
- [28] Byung-Gon Chun, Petros Maniatis, Scott Shenker, and John Kubiatowicz. 2007. Attested Append-Only Memory: Making Adversaries Stick to Their Word. *SIGOPS Oper. Syst. Rev.* 41, 6 (oct 2007), 189–204.
- [29] Allen Clement, Manos Kapritsos, Sangmin Lee, Yang Wang, Lorenzo Alvisi, Mike Dahlin, and Taylor Riche. 2009. Upright Cluster Services. In *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles (Big Sky, Montana, USA) (SOSP '09)*. Association for Computing Machinery, New York, NY, USA, 277–290.
- [30] James C. Corbett, Jeffrey Dean, Michael Epstein, Andrew Fikes, Christopher Frost, J. J. Furman, Sanjay Ghemawat, Andrey Gubarev, Christopher Heiser, Peter Hochschild, Wilson Hsieh, Sebastian Kanthak, Eugene Kogan, Hongyi Li, Alexander Lloyd, Sergey Melnik, David Mwaura, David Nagle, Sean Quinlan, Rajesh Rao, Lindsay Rolig, Yasushi Saito, Michal Szymaniak, Christopher Taylor, Ruth Wang, and Dale Woodford. 2012. Spanner: Google's globally-distributed database. In *Proceedings of the 10th USENIX conference on Operating Systems Design and Implementation (Hollywood, CA, USA) (OSDI'12)*. USENIX Association, Berkeley, CA, USA, 251–264.
- [31] James Cowling, Daniel Myers, Barbara Liskov, Rodrigo Rodrigues, and Liuba Shrira. 2006. HQ Replication: A Hybrid Quorum Protocol for Byzantine Fault Tolerance. In *Proceedings of the 7th Symposium on Operating Systems Design and Implementation (Seattle, Washington) (OSDI '06)*. USENIX Association, USA, 177–190.
- [32] T. Crain, C. Natoli, and V. Gramoli. 2021. Red Belly: A Secure, Fair and Scalable Open Blockchain. In *2021 IEEE Symposium on Security and Privacy (SP)*. IEEE Computer Society, Los Alamitos, CA, USA, 1501–1518.
- [33] George Danezis, Lefteris Kokoris-Kogias, Alberto Sonnino, and Alexander Spiegelman. 2022. Narwhal and Tusk: A DAG-Based Mempool and Efficient BFT Consensus. In *Proceedings of the Seventeenth European Conference on Computer Systems (Rennes, France) (EuroSys '22)*. Association for Computing Machinery, New York, NY, USA, 34–50.
- [34] Sourav Das, Zhuolun Xiang, Lefteris Kokoris-Kogias, and Ling Ren. 2023. Practical Asynchronous High-threshold Distributed Key Generation and Distributed Polynomial Sampling. In *32nd USENIX Security Symposium (USENIX Security 23)*. USENIX Association, Anaheim, CA, 5359–5376.
- [35] D. Dolev and H. R. Strong. 1983. Authenticated Algorithms for Byzantine Agreement. *SIAM J. Comput.* 12, 4 (1983), 656–666.

- [36] Jiaqing Du, Daniele Sciascia, Sameh Elnikety, Willy Zwaenepoel, and Fernando Pedone. 2014. Clock-RSM: Low-Latency Inter-Datacenter State Machine Replication Using Loosely Synchronized Physical Clocks. In *The 44th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*.
- [37] Sisi Duan, Michael K. Reiter, and Haibin Zhang. 2018. BEAT: Asynchronous BFT Made Practical. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (Toronto, Canada) (CCS '18)*. Association for Computing Machinery, New York, NY, USA, 2028–2041.
- [38] Vitor Enes, Carlos Baquero, Tuanir França Rezende, Alexey Gotsman, Matthieu Perrin, and Pierre Sutra. 2020. State-Machine Replication for Planet-Scale Systems. In *Proceedings of the Fifteenth European Conference on Computer Systems (Heraklion, Greece) (EuroSys '20)*. Association for Computing Machinery, New York, NY, USA, Article 24, 15 pages.
- [39] Ittay Eyal, Adem Efe Gencer, Emin Gun Sirer, and Robbert Van Renesse. 2016. Bitcoin-NG: A Scalable Blockchain Protocol. In *13th USENIX Symposium on Networked Systems Design and Implementation (NSDI 16)*. USENIX Association, Santa Clara, CA, 45–59.
- [40] Michael J. Fischer, Nancy A. Lynch, and Michael S. Paterson. 1985. Impossibility of distributed consensus with one faulty process. *J. ACM* 32, 2 (April 1985), 374–382.
- [41] Bryan Ford, Philipp Jovanovic, and Ewa Syta. 2020. Que Sera Consensus: Simple Asynchronous Agreement with Private Coins and Threshold Logical Clocks. *CoRR* abs/2003.02291 (2020). arXiv:2003.02291
- [42] Yossi Gilad, Rotem Hemo, Silvio Micali, Georgios Vlachos, and Nickolai Zeldovich. 2017. Algorand: Scaling Byzantine Agreements for Cryptocurrencies. In *Proceedings of the 26th Symposium on Operating Systems Principles (Shanghai, China) (SOSP '17)*. Association for Computing Machinery, New York, NY, USA, 51–68.
- [43] G. Golan Gueta, I. Abraham, S. Grossman, D. Malkhi, B. Pinkas, M. Reiter, D. Seredinschi, O. Tamir, and A. Tomescu. 2019. SBFT: A Scalable and Decentralized Trust Infrastructure. In *2019 49th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, 568–580.
- [44] Bingyong Guo, Zhenliang Lu, Qiang Tang, Jing Xu, and Zhenfeng Zhang. 2020. Dumbo: Faster Asynchronous BFT Protocols. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security (Virtual Event, USA) (CCS '20)*. Association for Computing Machinery, New York, NY, USA, 803–818.
- [45] Yue Guo, Rafael Pass, and Elaine Shi. 2019. Synchronous, with a Chance of Partition Tolerance. In *Advances in Cryptology – CRYPTO 2019*, Alexandra Boldyreva and Daniele Micciancio (Eds.). Springer International Publishing, Cham, 499–529.
- [46] Zhenyu Guo, Chuntao Hong, Mao Yang, Dong Zhou, Lidong Zhou, and Li Zhuang. 2014. Rex: Replication at the Speed of Multi-Core. In *Proceedings of the Ninth European Conference on Computer Systems (Amsterdam, The Netherlands) (EuroSys '14)*. Association for Computing Machinery, New York, NY, USA, Article 11, 14 pages.
- [47] Timo Hanke, Mahnush Movahedi, and Dominic Williams. 2018. DFINITY Technology Overview Series, Consensus System. *CoRR* abs/1805.04548 (2018). arXiv:1805.04548
- [48] Maurice P. Herlihy and Jeannette M. Wing. 1990. Linearizability: A Correctness Condition for Concurrent Objects. *ACM Trans. Program. Lang. Syst.* 12, 3 (July 1990), 463–492.
- [49] Xin Jin, Xiaozhou Li, Haoyu Zhang, Nate Foster, Jeongkeun Lee, Robert Soulé, Changhoon Kim, and Ion Stoica. 2018. NetChain: Scale-Free Sub-RTT Coordination. In *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*. USENIX Association, Renton, WA, 35–49.
- [50] Rüdiger Kapitza, Johannes Behl, Christian Cachin, Tobias Distler, Simon Kuhnle, Seyed Vahid Mohammadi, Wolfgang Schröder-Preikschat, and Klaus Stengel. 2012. CheapBFT: Resource-Efficient Byzantine Fault Tolerance. In *Proceedings of the 7th ACM European Conference on Computer Systems (Bern, Switzerland) (EuroSys '12)*. Association for Computing Machinery, New York, NY, USA, 295–308.
- [51] Manos Kapritsos, Yang Wang, Vivien Quema, Allen Clement, Lorenzo Alvisi, and Mike Dahlin. 2012. All about Eve: execute-verify replication for multi-core servers. In *Proceedings of the 10th USENIX conference on Operating Systems Design and Implementation (Hollywood, CA, USA) (OSDI'12)*. USENIX Association, Berkeley, CA, USA, 237–250.
- [52] Idit Keidar, Eleftherios Kokoris-Kogias, Oded Naor, and Alexander Spiegelman. 2021. All You Need is DAG. In *Proceedings of the 2021 ACM Symposium on Principles of Distributed Computing (Virtual Event, Italy) (PODC'21)*. Association for Computing Machinery, New York, NY, USA, 165–175.
- [53] Mahimna Kelkar, Fan Zhang, Steven Goldfeder, and Ari Juels. 2020. Order-Fairness for Byzantine Consensus. In *Advances in Cryptology – CRYPTO 2020*, Daniele Micciancio and Thomas Ristenpart (Eds.). Springer International Publishing, Cham, 451–480.
- [54] Marios Kogias and Edouard Bugnion. 2020. HovercRaft: Achieving Scalability and Fault-Tolerance for Microsecond-Scale Datacenter Services. In *Proceedings of the Fifteenth European Conference on Computer Systems (Heraklion, Greece) (EuroSys '20)*. Association for Computing Machinery, New York, NY, USA, Article 25, 17 pages.
- [55] Ramakrishna Kotla, Lorenzo Alvisi, Mike Dahlin, Allen Clement, and Edmund Wong. 2010. Zyzzyva: Speculative Byzantine Fault Tolerance. *ACM Trans. Comput. Syst.* 27, 4, Article 7 (jan 2010), 39 pages.
- [56] Leslie Lamport. 2001. Paxos Made Simple. *ACM SIGACT News* 32, 4 (December 2001).
- [57] Leslie Lamport. 2005. *Generalized Consensus and Paxos*. Technical Report MSR-TR-2005-33. 60 pages.
- [58] Leslie Lamport, Robert Shostak, and Marshall Pease. 1982. The Byzantine Generals Problem. *ACM Trans. Program. Lang. Syst.* 4, 3 (July 1982), 382–401.
- [59] Dave Levin, John R. Douceur, Jacob R. Lorch, and Thomas Moscibroda. 2009. TrInc: Small Trusted Hardware for Large Distributed Systems. In *6th USENIX Symposium on Networked Systems Design and Implementation (NSDI 09)*. USENIX Association, Boston, MA.
- [60] Yoad Lewenberg, Yonatan Sompolsky, and Aviv Zohar. 2015. Inclusive block chain protocols. In *International Conference on Financial Cryptography and Data Security*. Springer, 528–547.
- [61] Chenxing Li, Peilun Li, Dong Zhou, Zhe Yang, Ming Wu, Guang Yang, Wei Xu, Fan Long, and Andrew Chi-Chih Yao. 2020. A Decentralized Blockchain with High Throughput and Fast Confirmation. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*. USENIX Association, 515–528.
- [62] Jialin Li, Ellis Michael, Naveen Kr. Sharma, Adriana Szekeres, and Dan R. K. Ports. 2016. Just Say NO to Paxos Overhead: Replacing Consensus with Network Ordering. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*. USENIX Association, Savannah, GA, 467–483.
- [63] Chao Liu, Sisi Duan, and Haibin Zhang. 2020. EPIC: Efficient Asynchronous BFT with Adaptive Security. In *2020 50th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. 437–451.
- [64] Shengyun Liu, Paolo Viotti, Christian Cachin, Vivien Quéma, and Marko Vukolic. 2016. XFT: Practical Fault Tolerance beyond Crashes. In *Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation (Savannah, GA, USA) (OSDI'16)*. USENIX Association, USA, 485–500.
- [65] Shengyun Liu and Marko Vukolic. 2017. Leader Set Selection for Low-Latency Geo-Replicated State Machine. *IEEE Transactions on Parallel and Distributed Systems* 28, 7 (2017), 1933–1946.
- [66] Marta Likhava, Giuliano Losa, David Mazières, Graydon Hoare, Nicolas Barry, Eli Gafni, Jonathan Jove, Rafał Malinowsky, and Jed McCaleb. 2019. Fast and Secure Global Payments with Stellar. In *Proceedings of*

- the 27th ACM Symposium on Operating Systems Principles (Huntsville, Ontario, Canada) (SOSP '19). Association for Computing Machinery, New York, NY, USA, 80–96.
- [67] Yanhua Mao, Flavio P. Junqueira, and Keith Marzullo. 2008. Mencius: Building Efficient Replicated State Machines for WANs. In *Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation* (San Diego, California) (OSDI'08). USENIX Association, Berkeley, CA, USA, 369–384.
- [68] Andrew Miller, Yu Xia, Kyle Croman, Elaine Shi, and Dawn Song. 2016. The Honey Badger of BFT Protocols. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security* (Vienna, Austria) (CCS '16). Association for Computing Machinery, New York, NY, USA, 31–42.
- [69] Zarko Milosevic, Martin Biely, and André Schiper. 2013. Bounded Delay in Byzantine-Tolerant State Machine Replication. In *2013 IEEE 32nd International Symposium on Reliable Distributed Systems*. 61–70.
- [70] Iulian Moraru, David G. Andersen, and Michael Kaminsky. 2013. There is More Consensus in Egalitarian Parliaments. In *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles* (Farmington, Pennsylvania) (SOSP '13). Association for Computing Machinery, New York, NY, USA, 358–372.
- [71] Achour Mostefaoui, Hamouma Moumen, and Michel Raynal. 2014. Signature-Free Asynchronous Byzantine Consensus with $t < n/3$ and $o(n^2)$ Messages. In *Proceedings of the 2014 ACM Symposium on Principles of Distributed Computing* (Paris, France) (PODC '14). Association for Computing Machinery, New York, NY, USA, 2–9.
- [72] Satoshi Nakamoto. 2019. *Bitcoin: A peer-to-peer electronic cash system*. Technical Report. Manubot.
- [73] Ray Neiheiser, Miguel Matos, and Luís Rodrigues. 2021. Kauri: Scalable BFT Consensus with Pipelined Tree-Based Dissemination and Aggregation. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*. Association for Computing Machinery, New York, NY, USA, 35–48.
- [74] Haochen Pan, Jesse Tuglu, Neo Zhou, Tianshu Wang, Yicheng Shen, Xiong Zheng, Joseph Tassarotti, Lewis Tseng, and Roberto Palmieri. 2021. Rabia: Simplifying State-Machine Replication Through Randomization. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles* (Virtual Event, Germany) (SOSP '21). Association for Computing Machinery, New York, NY, USA, 472–487.
- [75] M. Pease, R. Shostak, and L. Lamport. 1980. Reaching Agreement in the Presence of Faults. *J. ACM* 27, 2 (apr 1980), 228–234.
- [76] Dan R. K. Ports, Jialin Li, Vincent Liu, Naveen Kr. Sharma, and Arvind Krishnamurthy. 2015. Designing Distributed Systems Using Approximate Synchrony in Data Center Networks. In *12th USENIX Symposium on Networked Systems Design and Implementation* (NSDI 15). USENIX Association, Oakland, CA, 43–57.
- [77] Ji Qi, Xusheng Chen, Yunpeng Jiang, Jianyu Jiang, Tianxiang Shen, Shixiong Zhao, Sen Wang, Gong Zhang, Li Chen, Man Ho Au, and Heming Cui. 2021. Bidl: A High-Throughput, Low-Latency Permissioned Blockchain Framework for Datacenter Networks. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles* (Virtual Event, Germany) (SOSP '21). Association for Computing Machinery, New York, NY, USA, 18–34.
- [78] Team Rocket, Maofan Yin, Kevin Sekniqi, Robbert van Renesse, and Emin Gün Sirer. 2019. Scalable and probabilistic leaderless BFT consensus through metastability. *arXiv preprint arXiv:1906.08936* (2019).
- [79] João Sousa, Alysson Bessani, and Marko Vukolic. 2018. A Byzantine Fault-Tolerant Ordering Service for the Hyperledger Fabric Blockchain Platform. In *2018 48th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. 51–58.
- [80] Chrysoula Stathakopoulou, Matej Pavlovic, and Marko Vukolić. 2022. State Machine Replication Scalability Made Simple. In *Proceedings of the Seventeenth European Conference on Computer Systems* (Rennes, France) (EuroSys '22). Association for Computing Machinery, New York, NY, USA, 17–33.
- [81] Florian Suri-Payer, Matthew Burke, Zheng Wang, Yunhao Zhang, Lorenzo Alvisi, and Natacha Crooks. 2021. Basil: Breaking up BFT with ACID (Transactions). In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*. Association for Computing Machinery, New York, NY, USA, 1–17.
- [82] Lei Yang, Seo Jin Park, Mohammad Alizadeh, Sreeram Kannan, and David Tse. 2022. DispersedLedger: High-Throughput Byzantine Consensus on Variable Bandwidth Networks. In *19th USENIX Symposium on Networked Systems Design and Implementation* (NSDI 22). USENIX Association, Renton, WA, 493–512.
- [83] Maofan Yin, Dahlia Malkhi, Michael K. Reiter, Guy Golan Gueta, and Ittai Abraham. 2019. HotStuff: BFT Consensus with Linearity and Responsiveness. In *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing* (Toronto ON, Canada) (PODC '19). Association for Computing Machinery, New York, NY, USA, 347–356.
- [84] Yunhao Zhang, Srinath Setty, Qi Chen, Lidong Zhou, and Lorenzo Alvisi. 2020. Byzantine Ordered Consensus without Byzantine Oligarchy. In *14th USENIX Symposium on Operating Systems Design and Implementation* (OSDI 20). USENIX Association, 633–649.